PROJECT PLAN: Electronics for Robotics Workshop Series

	Title	Description
Session 1	Intro to microcontrollers	Arduino, ESP32, STM32, ARM architecture, C++ programming basics, loops and control structures LED patterns, Serial monitor
Session 2	Sensor inputs	Push buttons, Pull-ups, Interrupts, sensor readings, analog inputs
Session 3	Motors and PWM	PWM, DC and AC motors, Brushless and Brushed motors, H-Bridge, ESCs
Session 4	Communication protocols	Serial protocols theory, I2C, SPI, CAN protocol, Bluetooth and WIFI protocols
Session 5	PID control theory	Control theory, on-off control, PID for line following, algorithm implementation

Blink

```
void setup() {
 // initialize digital pin LED_BUILTIN as an output.
 pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
 digitalWrite(LED_BUILTIN, HIGH);  // turn the LED on (HIGH
is the voltage level)
 delay(1000);                      // wait for a second
 digitalWrite(LED_BUILTIN, LOW);   // turn the LED off by
making the voltage LOW
 delay(1000);                      // wait for a second
}
```

Fade

```
int led = 9;           // the PWM pin the LED is attached to
int brightness = 0;    // how bright the LED is
int fadeAmount = 5;    // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
 // declare pin 9 to be an output:
 pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
 // set the brightness of pin 9:
 analogWrite(led, brightness);

 // change the brightness for next time through the loop:
 brightness = brightness + fadeAmount;

 // reverse the direction of the fading at the ends of the
fade:
 if (brightness <= 0 || brightness >= 255) {
  fadeAmount = -fadeAmount;
 }
 // wait for 30 milliseconds to see the dimming effect
 delay(30);
}
```

Serial monitor

```
void setup() {
// Start serial communication at 9600 baud
Serial.begin(9600);

Serial.println("Hello, World!");
Serial.println("Arduino Serial Monitor Demo");
Serial.println("--------------------------");
}

void loop() {
Serial.print("Counter = ");
Serial.println(counter);

counter++; // Increase counter by 1

delay(1000); // Wait 1 second
}
```

push button readings

```
void setup() {
 // initialize serial communication at 9600 bits per
second:
 Serial.begin(9600);
 // make the pushbutton's pin an input:
 pinMode(pushButton, INPUT);
}

// the loop routine runs over and over again forever:
void loop() {
 // read the input pin:
 int buttonState = digitalRead(pushButton);
 // print out the state of the button:
 Serial.println(buttonState);
 delay(1);  // delay in between reads for stability
}
```

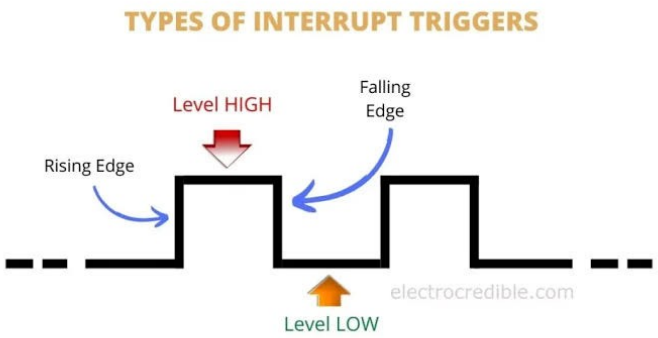
Push button to LED

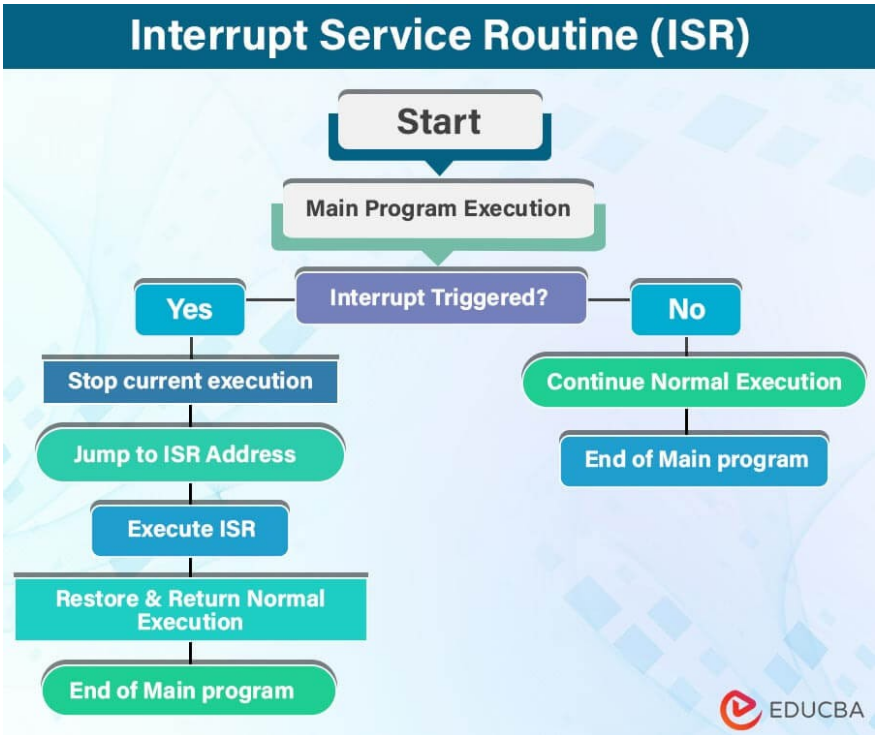
```
const int buttonPin = 2;
const int ledPin = 13;

int buttonState = 0;

void setup() {
pinMode(buttonPin, INPUT_PULLUP); // Internal pull-up
resistor
pinMode(ledPin, OUTPUT);

Serial.begin(9600);
}

void loop() {
buttonState = digitalRead(buttonPin);

// Because of INPUT_PULLUP:
// NOT pressed = HIGH
// Pressed = LOW

if (buttonState == LOW) {
digitalWrite(ledPin, HIGH); // Turn LED ON
Serial.println("Button Pressed - LED ON");
} else {
digitalWrite(ledPin, LOW); // Turn LED OFF
Serial.println("Button Released - LED OFF");
}

delay(100); // small delay for stability
}
```

Interrupts

```
const int buttonPin = 2;
const int ledPin = 13;

volatile bool ledState = false; // shared between ISR and main code

void setup() {
```

```
pinMode(buttonPin, INPUT_PULLUP);
pinMode(ledPin, OUTPUT);

Serial.begin(9600);

// Attach interrupt
attachInterrupt(digitalPinToInterrupt(buttonPin), buttonPressed, FALLING);

Serial.println("Interrupt Demo Started");
}

void loop() {
// Main loop does nothing important
digitalWrite(ledPin, ledState);

// Just slow printing to see it's not polling
Serial.println("Loop running...");
delay(1000);
}

// Interrupt Service Routine (ISR)
void buttonPressed() {
ledState = !ledState; // toggle LED state
}
```





Analog Inputs

```
const int potPin = A0;

int potValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  potValue = analogRead(potPin);

  Serial.print("Potentiometer Value: ");
  Serial.println(potValue);

  delay(100);
}
```

Analog Inputs Voltage

```
const int potPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int value = analogRead(potPin);

  float voltage = value * 5.0 / 1023.0;

  Serial.print("ADC = ");
  Serial.print(value);

  Serial.print("    Voltage = ");
  Serial.print(voltage);
  Serial.println(" V");

  delay(100);
```

```
}
```

Analog input control LED example

```
const int potPin = A0;
const int ledPin = 13;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int potValue = analogRead(potPin);

  digitalWrite(ledPin, HIGH);
  delay(potValue);

  digitalWrite(ledPin, LOW);
  delay(potValue);
}
```

LED Pattern Examples

Knight Rider / Scanner Effect (VERY popular)

```
const int ledPins[] = {2, 3, 4, 5, 6};
const int numLeds = 5;
void setup() {
  for (int i = 0; i < numLeds; i++) {
    pinMode(ledPins[i], OUTPUT);
  }
}
void loop() {
  // Left to right
  for (int i = 0; i < numLeds; i++) {
    digitalWrite(ledPins[i], HIGH);
    delay(100);
    digitalWrite(ledPins[i], LOW);
  }
  // Right to left
  for (int i = numLeds - 1; i >= 0; i--) {
    digitalWrite(ledPins[i], HIGH);
    delay(100);
```

```
    digitalWrite(ledPins[i], LOW);
  }
}
```

Running Lights (One by one ON)

```
const int leds[] = {2, 3, 4, 5};
const int n = 4;
void setup() {
  for (int i = 0; i < n; i++) {
    pinMode(leds[i], OUTPUT);
  }
}
void loop() {
  for (int i = 0; i < n; i++) {
    digitalWrite(leds[i], HIGH);
    delay(200);
    digitalWrite(leds[i], LOW);
  }
}
```

Ping-Pong Pattern (Back and forth glow)

```
const int leds[] = {2, 3, 4, 5};
const int n = 4;
void setup() {
  for (int i = 0; i < n; i++) {
    pinMode(leds[i], OUTPUT);
  }
}
void loop() {
  for (int i = 0; i < n; i++) {
    digitalWrite(leds[i], HIGH);
    delay(150);
    digitalWrite(leds[i], LOW);
  }
  for (int i = n - 2; i > 0; i--) {
    digitalWrite(leds[i], HIGH);
    delay(150);
    digitalWrite(leds[i], LOW);
  }
}
```

Custom loops

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  blink(3);  // blink 3 times
  delay(1000);
}

void blink(int numTimes) {
  for (int i = 0; i < numTimes; i++) {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(500);
    digitalWrite(LED_BUILTIN, LOW);
    delay(500);
  }
}
```

Distance Measurements using SR04

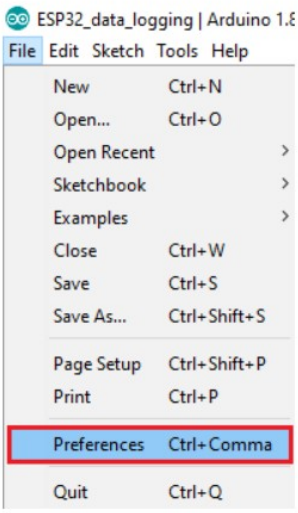
```
// HC-SR04 Ultrasonic Distance Sensor - No Library Example

const int trigPin = 9;
const int echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  // Make sure trigger pin is low first
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Send a 10 microsecond pulse to trigger
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read the time (in microseconds) for echo pin to go high then low
  long duration = pulseIn(echoPin, HIGH);

  // Convert duration to distance
  // Speed of sound = 343 m/s = 0.0343 cm/microsecond
  // Divide by 2 because sound travels to object and back
  float distanceCm = (duration * 0.0343) / 2;

  Serial.print("Distance: ");
  Serial.print(distanceCm);
  Serial.println(" cm");

  delay(500); // wait half a second between readings
}
```

HC-SR04 Distance Measurement - Using NewPing Library

```
#include <NewPing.h>

#define TRIG_PIN 9
#define ECHO_PIN 10
#define MAX_DISTANCE 400  // Maximum distance to measure (in cm)

NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

void setup() {
  Serial.begin(9600);
}

void loop() {
  delay(500); // wait between readings (avoids sensor interference)

  unsigned int distanceCm = sonar.ping_cm();

  Serial.print("Distance: ");
  Serial.print(distanceCm);
  Serial.println(" cm");
}
```

To install the ESP32 board in your Arduino IDE, follow these next instructions:

1. In your Arduino IDE, go to **File> Preferences**



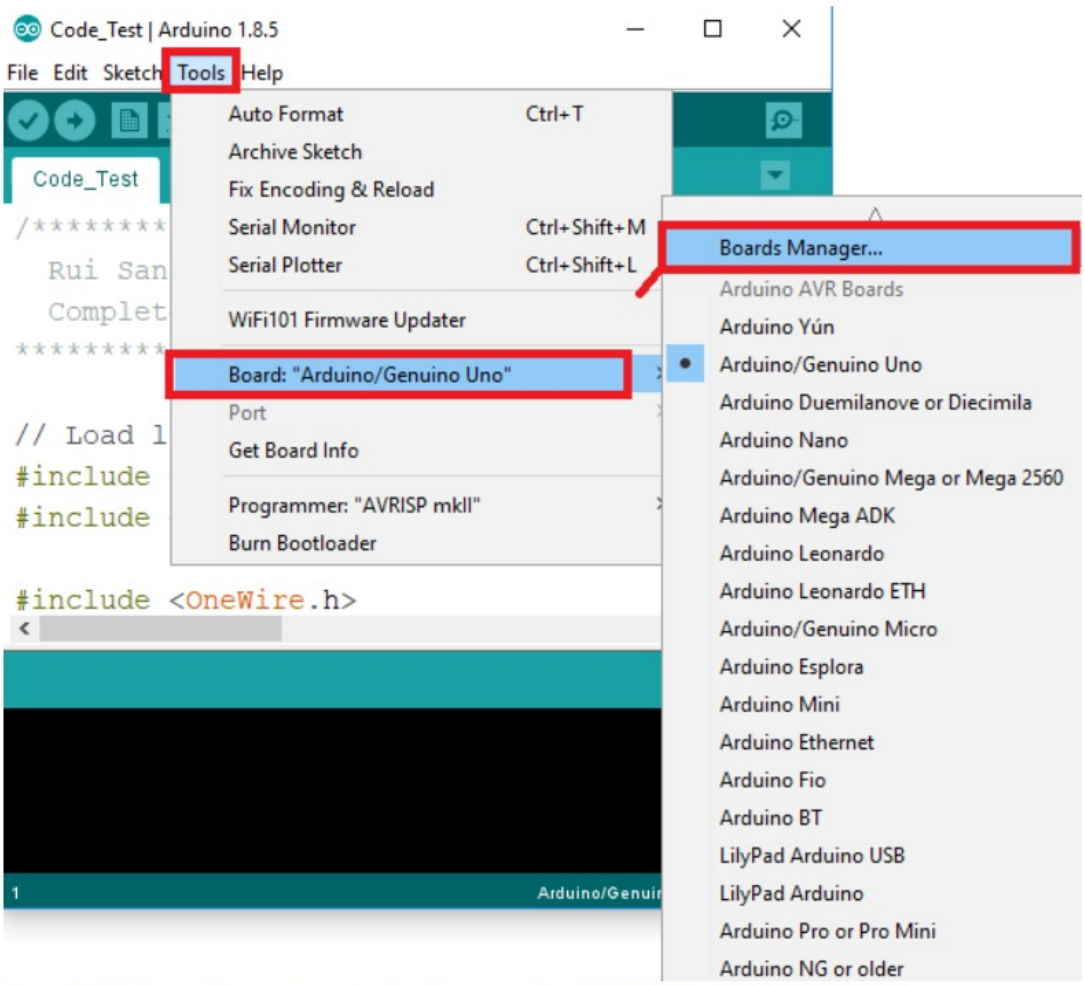
2. Enter the following into the “Additional Board Manager URLs” field:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-
pages/package_esp32_index.json
```

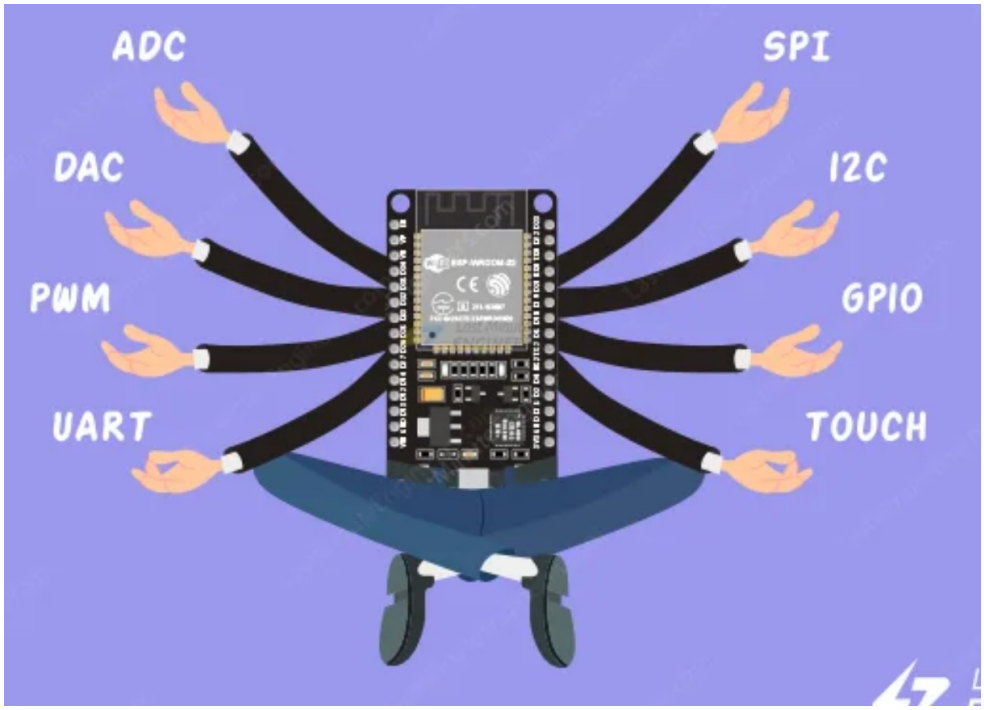
Enter the following into the “Additional Board Manager URLs” field:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Open the Boards Manager. Go to **Tools** > **Board** > **Boards Manager…**



4. Search for **ESP32** and press install button for the "**ESP32 by Espressif Systems**":



Arduino Uno vs ESP32:
Which One Should You Choose?

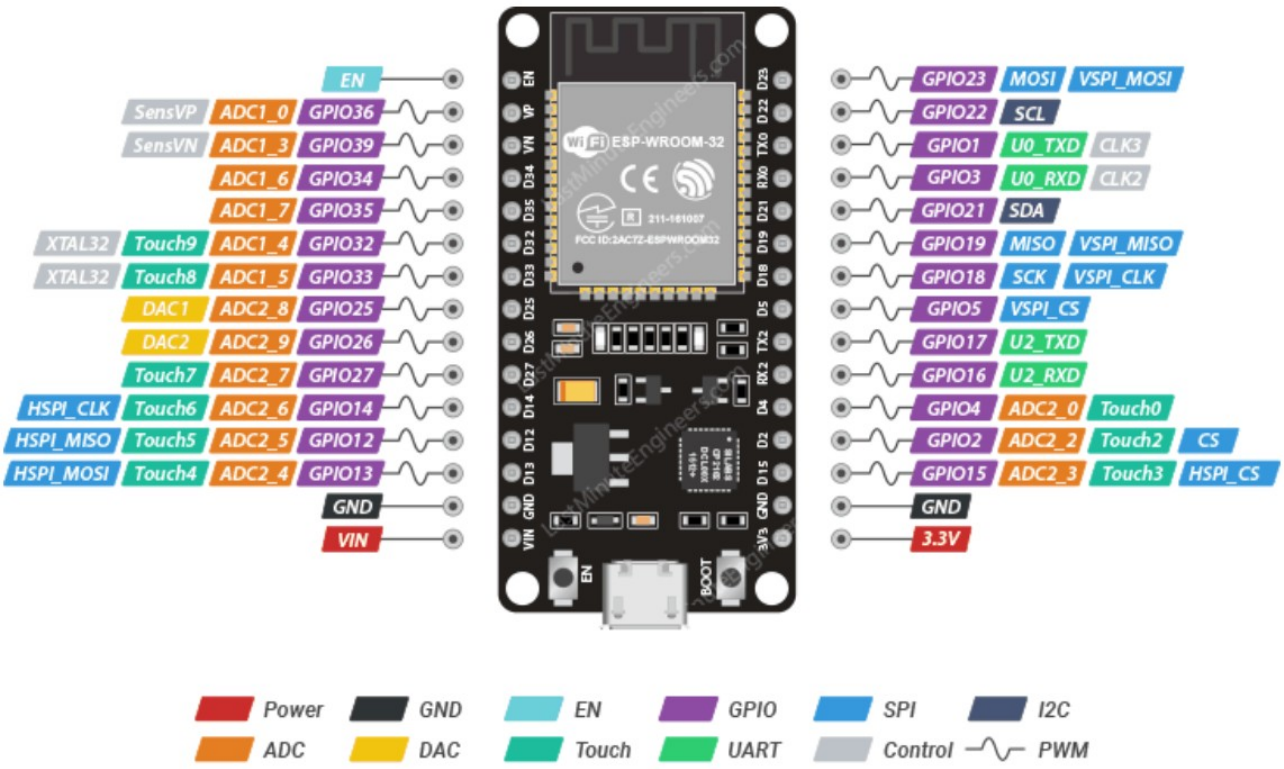
	Arduino Uno	ESP32
	ATmega328P	Dual-core Tensilica Xtensa LX6
	32 KB	4 MB (varies by model)
	2 KB	520 KB SRAM
	16 MHz	Up to 240 MHz
	No-buitt-ilin Wi-Fi/Blut	34+ GPIOs
	5V	3,3 V
Price Range	$5-$10	$5-$15
	Huge community & shields	Growing community, more bult-in features

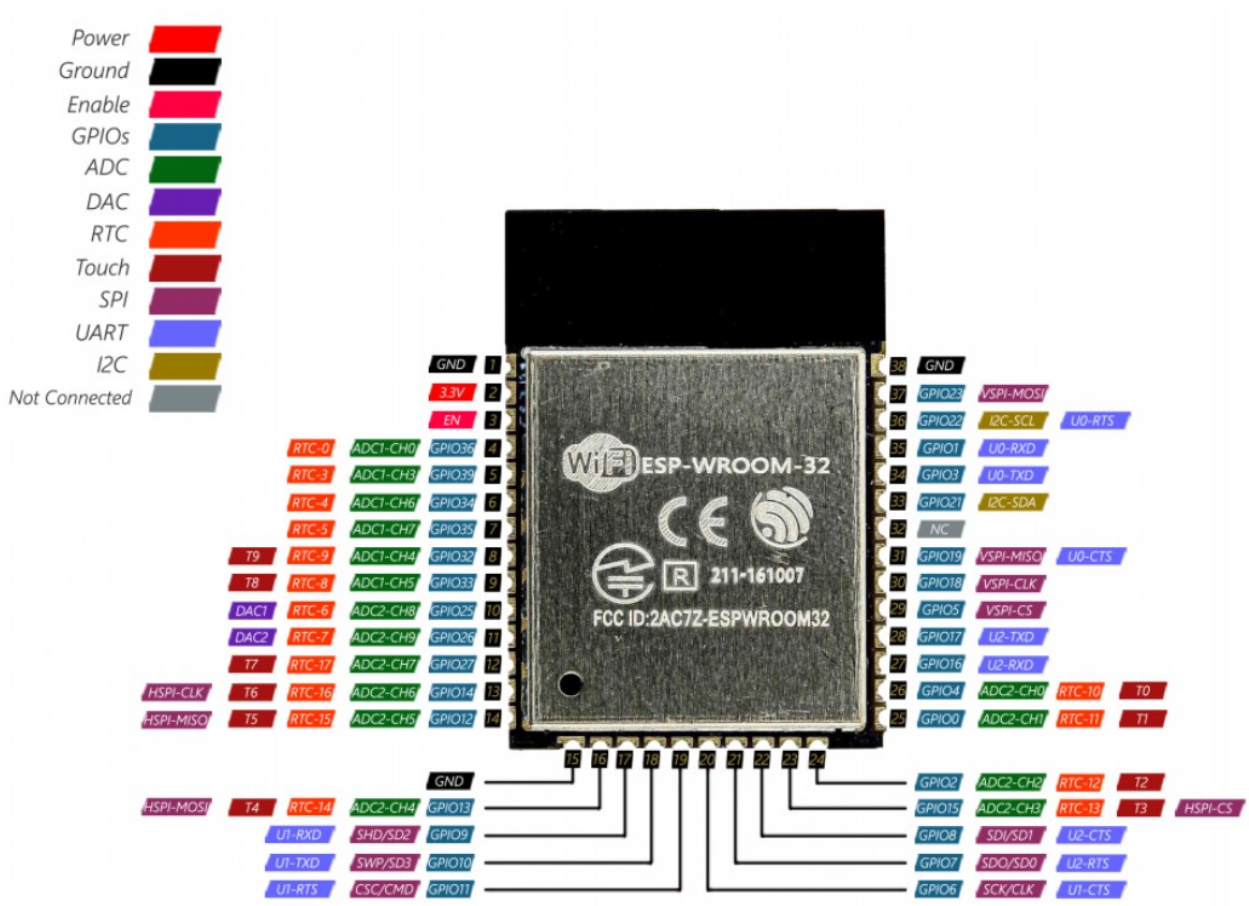
Choose Arduino Uno if:
You're a beginner, need simple ttams or who learn microcontroller basics.

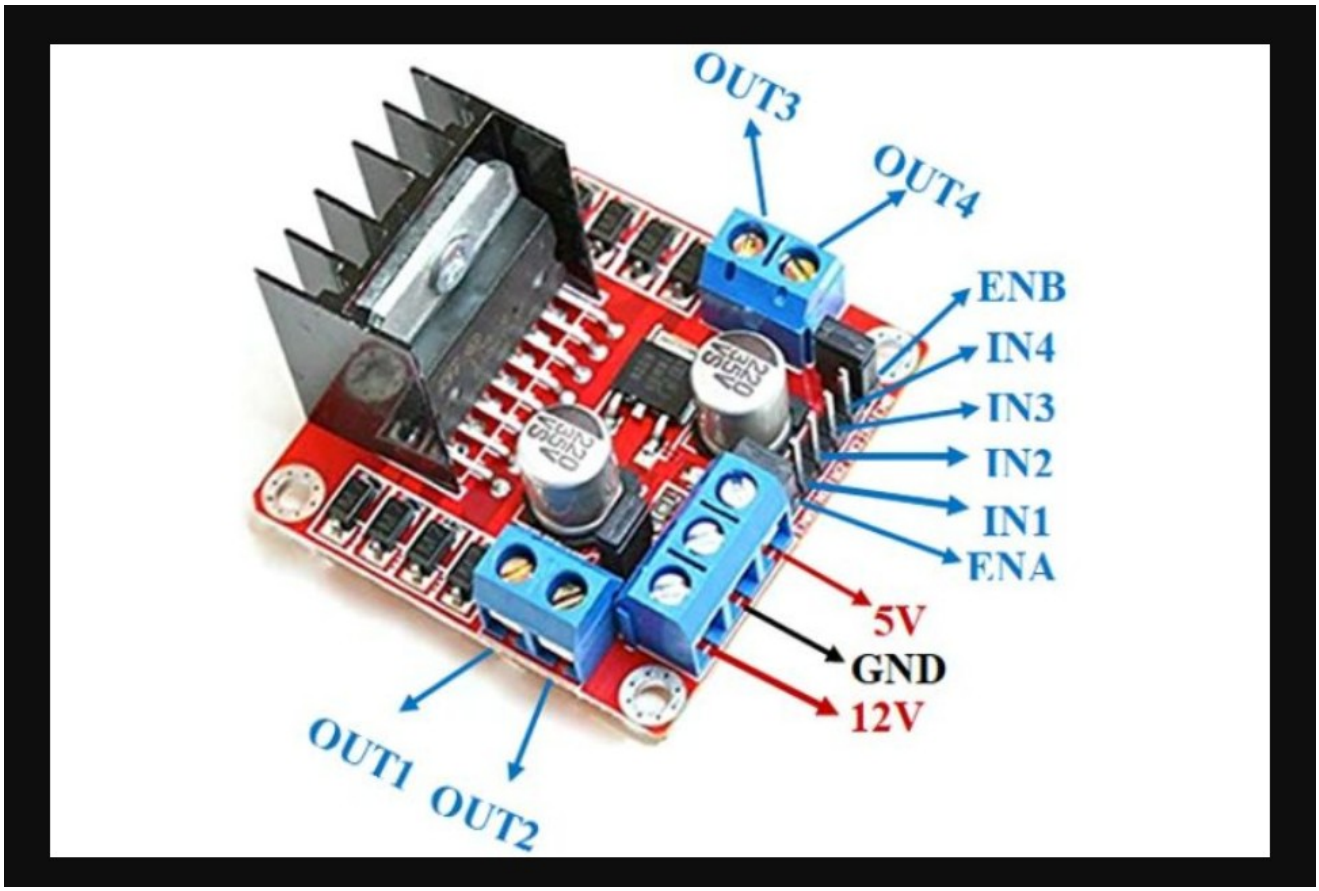
Choose ESP32 if:
You need Wi-Fi/Blutooth, more processing power, or want to build amved loT projects.

My Take: ESP32 is like "smartphone" of microcotrollers versatile powerful and cost-effective. But the Arduino Uno is still an excellent

#loT #Arduino #ESP32 #EmbeddedSystems #Microcontrollers #Innovation







L298N H-Bridge - ESP32 Simple Forward Drive

```
// Motor A (Left)
#define IN1 18
#define IN2 19
#define ENA 22

// Motor B (Right)
#define IN3 16
#define IN4 17
#define ENB 23

int speed = 200; // 0-255

void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);

  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
}

void loop() {
  // Motor A forward
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, speed);
```

```
  // Motor B forward
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, speed);
}
```

Square try

```
// Motor A (Left)
#define IN1 18
#define IN2 19
#define ENA 22

// Motor B (Right)
#define IN3 16
#define IN4 17
#define ENB 23

int speed = 200; // 0-255

void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);

  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
}

void loop() {
  // Drive in a square: 4 sides, turn right at each corner
  for (int i = 0; i < 4; i++) {
    forward();
    delay(1000);   // adjust for side length

    stopCar();
    delay(300);

    turnRight();
    delay(400);    // adjust for a ~90 degree turn

    stopCar();
    delay(300);
  }

  stopCar();
  while (true); // stop here after one square (remove this line to repeat)
}

// --- Movement functions ---

void forward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
```

```
}

void backward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}

void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}

void stopCar() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```